

Property Inference Attacks on Fully Connected Neural Networks using Permutation Invariant Representations

2018 ACM CCS ——属性推断攻击经典白盒方案深度解析

文献分享与复现

2026 年 5 月 4 日

目录 (Agenda)

1. 威胁模型与攻击直觉 (Threat Model)
2. 核心挑战：深扒排列不变性 (Permutation Invariance)
3. 破局之道：设计排列不变表示
4. 攻击流水线与实验深度分析
5. 复现计划与展望

1.1 重新认识属性推断 (Property Inference)

核心定义：模型所有者发布了一个训练好的模型，攻击者试图通过分析模型参数，反向推导出训练集中未公开的宏观统计分布（通常与模型主任务无关）。

隐私攻击的“宏观”与“微观”

- **微观级别 (Micro-level):** 成员推断 (Membership)、模型逆向 (Model Inversion)。它们关心的是**特定的个体**（如：张三的数据是否被用了？）。
- **宏观级别 (Macro-level):** 属性推断 (Property/Distribution)。它关心的是**群体的统计特征**（如：训练集中女性占比是否超过 60%？）。

为什么危害巨大？可能泄露企业的核心商业机密（如客户群体的受众画像、数据采集环境），或用于审计模型是否使用了存在偏见/违规的数据集。

1.2 论文的威胁模型设定 (White-box Assumption)

攻击者的能力 (Capabilities):

- **白盒访问 (White-box):** 攻击者拥有目标模型 (Target Model) 的完整架构和**所有权重参数 (Weights & Biases)**。
- **辅助数据 (Auxiliary Data):** 攻击者拥有与受害者训练集分布相似的辅助数据集，用于后续构造影子模型。

攻击者的目标 (Objective):

- 判断目标模型的训练集 D 是否具备某一特定属性 P (二分类问题)，或者推断该属性的具体比例 (回归问题)。

2.1 朴素的白盒攻击：展平基线 (Flatten Baseline)

直觉出发点：在完全白盒的假设下，攻击者拥有目标模型的所有权重 (Weights) 和偏置 (Biases)。最直接、最符合直觉的攻击方式是什么？

提取全部参数 → 展平为一维向量 → 训练二分类元模型 (Meta-Classifier)

论文中的定义：作者将其作为**基线模型 (Baseline)**。也就是把网络所有层的参数 $W_1, b_1, W_2, b_2 \dots$ 简单粗暴地 concatenate 成一个超长向量 $V \in \mathbb{R}^d$ 。

直觉的崩塌 (Why it fails?)

实验表明，这种朴素的 Flatten 方法在推断宏观属性时，准确率几乎等同于随机猜测 ($\sim 50\%$)。

原因：深度学习优化的随机性（初始化、SGD 打乱），导致即使是在完全相同的分布上训练出的两个模型，其内部也会产生巨大的“位置噪音” (Position Noise)。

2.2 直觉的崩塌：Flatten 失败的三大元凶

- 1. 欧氏空间的“距离欺骗”：

- 元分类器判断两个样本是否同类的依据是高维特征空间中的欧氏距离。
- 若模型 A 的关键特征由第 1 个神经元负责，模型 B 由第 100 个负责，展平后这两者的距离极其遥远，导致元分类器被表象欺骗。

- 2. 恐怖的阶乘爆炸 ($n!$):

- 仅一个含 100 个神经元的隐藏层，就有 $100!$ 种 (约 9.3×10^{157}) 等效排列。
- 同一“属性信号”在展平空间中会产生 $100!$ 个影子变体。仅靠几百个影子模型训练无异于盲人摸象，根本无法泛化。

- 3. 信号被噪音彻底淹没：

- “权重在哪个位置”（排列噪音）占据了巨大的数值方差，而“权重代表什么属性”（真实信号）仅引起微小的波动。
- 犹如在震耳欲聋的重金属音乐中，试图听清一只蚊子的叫声。

2.3 致命痛点：同功不同构 (Permutation Invariance)

如图所示的神经元互换：

- 假设我们交换隐藏层的两个节点位置。
- 其对应的入边权重和出边权重会发生交叉变换。
- **结果：**尽管内部的权重矩阵大变样，但网络最终的前向预测输出 y 丝毫不受影响。

这也意味着如果直接展平权重，巨大的“位置噪音”会彻底淹没微弱的属性信号。

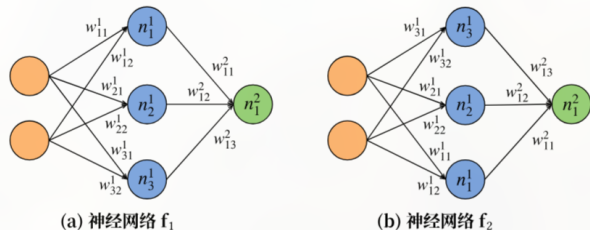


图 2：置换等价神经网络示例。神经网络 f_2 是通过将 f_1 中的神经元 n_1^1 和神经元 n_3^1 交换位置得到的。

2.4 数学推导 (上): 同功不同构的根源

为了理解“位置噪音”的破坏力，我们从前向传播的数学公式入手。
对于一个全连接网络，第 i 隐藏层的前向计算如下：

$$h^{(i)} = \sigma \left(W^{(i)} h^{(i-1)} + b^{(i)} \right) \quad (1)$$

引入排列操作：假设该层有 n 个神经元，我们引入一个 $n \times n$ 的排列矩阵 (Permutation Matrix) P 。 P 的特性是正交阵，即 $P^T P = P P^T = I$ 。

现在，由于训练的随机性，第 i 层的神经元位置被打乱了（本质是对向量 $h^{(i)}$ 进行行交换）：

$$\tilde{h}^{(i)} = P h^{(i)} = P \cdot \sigma \left(W^{(i)} h^{(i-1)} + b^{(i)} \right) = \sigma \left((P W^{(i)}) h^{(i-1)} + (P b^{(i)}) \right) \quad (2)$$

初步结论：一次神经元的互换，导致当前层的权重变成了 $\tilde{W}^{(i)} = P W^{(i)}$ ，偏置变成了 $\tilde{b}^{(i)} = P b^{(i)}$ 。由于矩阵左乘，这表现为行交换 (Row Swapping)。

2.5 数学推导 (下): 联动补偿与参数雪崩

但是, 网络是一个整体。第 i 层神经元被打乱了, 下一层 ($i+1$ 层) 接收到的输入变成了 $\tilde{h}^{(i)}$ 。为了保证网络最终的分类输出完全不变, 第 $i+1$ 层的权重必须进行补偿:

$$\text{原始下一层前向: } W^{(i+1)} h^{(i)} \quad (3)$$

$$\text{打乱后的下一层前向: } \tilde{W}^{(i+1)} \tilde{h}^{(i)} = \tilde{W}^{(i+1)} (Ph^{(i)}) \quad (4)$$

为了使两者相等: $\tilde{W}^{(i+1)} Ph^{(i)} = W^{(i+1)} h^{(i)}$, 等式右侧巧妙地插入 $I = P^T P$:

$$\tilde{W}^{(i+1)} Ph^{(i)} = W^{(i+1)} (P^T P) h^{(i)} = (W^{(i+1)} P^T) (Ph^{(i)}) \quad (5)$$

由此得出 $\tilde{W}^{(i+1)} = W^{(i+1)} P^T$ 。由于矩阵右乘, 这表现为列交换 (Column Swapping)。

灾难性后果: 一次简单的神经元位置互换 P , 会导致当前层权重行交换, 下一层权重列交换。功能完全等效的模型, 在参数空间上已经“面目全非”!

2.6 聚焦属性推断：困境与破局目标

现在我们将数学推导映射回属性推断 (PIA) 场景：

- **Meta-Classifer 的梦魇**：刚才的数学推导证实了，模型权重的展平 (Flatten) 包含了错综复杂的行/列交换。
- 面对如此巨大的排列空间，原本代表微弱属性分布的特征信号，被噪音产生了不可逾越的鸿沟（距离欺骗）。

我们需要做什么？(Our Goal)

我们必须设计一种映射函数（或特征表示） $\mathcal{F}(\text{weights})$ ，使得无论内部经历怎样的排列矩阵 P 变换，其输出的特征都是唯一且恒定的：

$$\mathcal{F}(W^{(i)}, W^{(i+1)}) \equiv \mathcal{F}(PW^{(i)}, W^{(i+1)}P^T)$$

这也就是接下来要介绍的核心技术：
排列不变表示 (Permutation Invariant Representation)

3.1 解决方案一：强制节点排序 (Canonical Form)

核心思路：人为制定规则，把打乱的神经元强行“对齐”。

算法 1 深度拆解：

- **计算指标**：遍历第 t 层，计算节点特征 $\mathcal{M}$ (如权范数) 并得出排序索引 σ (行 4-6)。
- **重排与联动补偿 (关键)**：
 - 行 7: $h'_t \leftarrow \sigma(h_t)$ ，对当前层的神元进行行交换。
 - 行 9: $w_{j*}^{t+1} \leftarrow \sigma(w_{j*}^{t+1})$ ，必须对下一层的对应权重进行列交换，以保证网络功能等效！
- **局限**：严重依赖启发式指标，网络较深时容易产生排序抖动。

Algorithm 1: Neuron Sorting

Input: A neural network f , a metric function $\mathcal{M}$

Output: Sorted feature representation $\mathcal{F}$

```
1  $\mathcal{F} \leftarrow []$ 
2 for  $t \leftarrow 1$  to  $|f| - 1$  do
3    $metrics \leftarrow []$ 
4   for  $i \leftarrow 1$  to  $|h_t|$  do
5      $metrics \leftarrow metrics || \mathcal{M}(n_i^t)$  // Get metrics to sort layer
6    $\sigma \leftarrow \text{argsort}(metrics)$ 
7    $h'_t \leftarrow \sigma(h_t)$ ; // Sort layer  $h_t$ 
8   for  $j \leftarrow 1$  to  $|h_{t+1}|$  do
9      $w_{j*}^{t+1} \leftarrow \sigma(w_{j*}^{t+1})$ ; // Permute next layer weights
10   $\mathcal{F} \leftarrow \mathcal{F} || \text{flatten}(h'_t)$ 
11  $\mathcal{F} \leftarrow \mathcal{F} || \text{flatten}(h_{|f|})$ 
12 return  $\mathcal{F}$ 
```

3.2 解决方案二：引入 DeepSets 抹除顺序

核心思路：既然排不好序，就干脆把网络层视为“无序集合 (Set)”。

Zaheer 等人的 DeepSets 理论证明：任何定义在集合上的排列不变函数，都可以分解为：

$$f(X) = \rho \left(\sum_{x \in X} \phi(x) \right) \quad (6)$$

- $\phi(x)$ ：特征提取网络，独立作用于集合中的每个元素。
- $\sum$ ：**对称聚合操作**。利用加法交换律 ($a + b = b + a$) 彻底抹除元素的顺序！
- $\rho(\cdot)$ ：读出网络，接收聚合后的总体特征，输出属性预测。

3.3 DeepSets 架构 workflow (Figure 5)

化矩阵为“无序集合”:

- ① **Flattening**: 提取节点 i 的入边权重 w_i 、偏置 b_i 及其上下文打包为一个独立向量。
- ② **Processing (ϕ)**: 通过 ϕ 网络独立提取节点特征。
- ③ **Summation (Σ)**: **核心魔法!** 对同层所有节点特征进行求和。由于 $A + B = B + A$, 强行抹除了节点的排列顺序噪音。
- ④ **Prediction (ρ)**: 拼接各层特征 L_i 送入 ρ 分类器预测最终属性 $P/\bar{P}$ 。

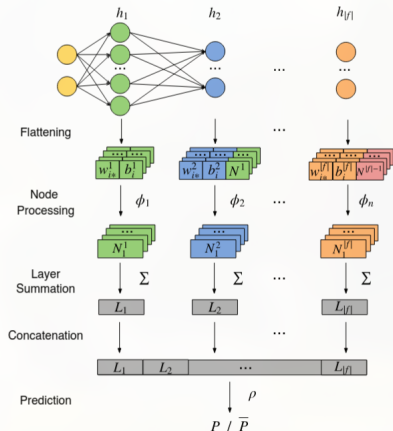


Figure 5: The workflow showing how the meta-classifier processes and learns the set-based representation of a neural network using the DeepSets architecture.

3.4 代码化理解：DeepSets 算法流程

算法 2 与架构图的对应关系：

- **特征提取 (行 5-6)：** 遍历每层的节点 i ，将其参数喂入 ϕ_t 获取节点特征 N^t 。
(对应图 5 的 *Node Processing*)
- **对称聚合 (行 7)：** $L_t \leftarrow \sum N_i^t$ ，这就是图中的 Layer Summation，实现排列不变性的关键步骤。
- **最终预测 (行 8-9)：** 拼接所有层的集合特征 $\mathcal{F}$ ，通过读出网络 $\rho(\mathcal{F})$ 输出预测结果。
(对应图 5 的 *Prediction*)

Algorithm 2: Process and learn the set-based representation.

Input: A neural network f , function ϕ_t for each layer h_t ,
function ρ

Output: P or $\bar{P}$

```
1  $\mathcal{F} \leftarrow []$ 
2  $N^0 \leftarrow []$ 
3 for  $t \leftarrow 1$  to  $|f|$  do
4    $N^t \leftarrow []$ 
5   for  $i \leftarrow 1$  to  $|h_t|$  do
6      $N^t \leftarrow N^t \parallel \phi_t(w_{i*}^t, b_i^t, N^{t-1});$  // Get node features
7    $L_t \leftarrow \sum_i N_i^t;$  // Obtain layer features
8    $\mathcal{F} \leftarrow \mathcal{F} \parallel L_t;$  // Obtain classifier features
9 return  $\rho(\mathcal{F});$  // Use  $\mathcal{F}$  to predict  $P/\bar{P}$ 
```

4.1 影子攻击全景流水线 (Shadow Training Pipeline)

攻击的实施是一个“元学习 (Meta-learning)”过程。

[请截取论文中整体的 Attack Pipeline/Architecture 图]

讲解重点 (结合图片指引):

1. **Data Generation:** 划分带属性 (D_1) 和不带属性 (D_0) 的辅助数据集。
2. **Shadow Training:** 批量训练大量结构相同的影子模型 (Shadow Models)。
3. **Meta Training:** 提取权重 -> 经过 DeepSets 提纯 -> 训练二分类元模型。
4. **Inference:** 对 Target Model 进行一键推断。

4.2 实验结果：降维打击 (Evaluation: Baseline vs. DeepSets)

[请截取论文中展示准确率对比的表格 (Tables) 或图表]

讲解重点分析：

- **Flatten Baseline:** 准确率往往徘徊在 50%-60% (基本等同于瞎猜)。
- **Node Sorting:** 准确率显著提升至 75%-85%，证明解决排列问题是有效的。
- **DeepSets:** 达到 85%-100% 的极高准确率，且对影子模型数量的需求更小！

4.3 深度案例一：微笑检测泄露颜值 (Case Study 1)

背景：目标模型是一个图片二分类器（任务：检测人脸是否在微笑）。

隐藏属性：训练集人群的整体吸引力比例 (Attractiveness)。

[请截取论文中关于 Attractiveness 的实验结果图 (如分布/ROC)]

解读：展示即便目标模型仅学习“嘴角的弧度”，它的权重深处依然记忆了数据集中人脸的整体面貌特征。攻击者借此可以进行 Fairness 审计或窃取隐私。

4.4 深度案例二：恶意软件检测泄露宿主环境 (Case Study 2)

背景：基于硬件性能计数器 (HPC) 的恶意软件检测模型。

隐藏属性：数据采集时的底层操作系统，是否打上了防御幽灵 (Spectre) 和熔断 (Meltdown) 漏洞的安全补丁。

安全启示：

- 模型的参数不仅是对“任务”的总结，更是对“数据环境”的快照。
- 共享看似无害的模型权重，等同于无意间共享了高度敏感的底层系统配置信息。

5. 组会代码复现展示计划 (Live Demo Plan)

为了让大家直观感受攻击威力，我将在后续环节展示 Pytorch 代码级别的复现：

- ① **数据集切分**：现场使用表格数据构造不同属性分布的 Shadow Datasets。
- ② **影子模型工厂**：运行脚本，自动化训练数百个 2-layer FCNN。
- ③ **算法对比对抗**：
 - 运行 Flatten Baseline 攻击（展示其失败）。
 - 运行 Node Sorting 攻击（展示性能提升）。
 - 运行 DeepSets 攻击架构（展示最终的降维打击效果）。

Q & A

感谢大家聆听！

(接下来进入代码复现环节...)